

# QUERY CLASSIFIER

Erik Halldorson, Praneel Murali, Andrew Nguyen, Chloë Posluszny, Chenhe Yuan

## Introduction

This project aims to classify and group example queries by their semantic similarity across different categories. The dataset was created by prompting a large language model (LLM) to generate a variety of realistic, natural-language questions that users might ask about topics such as technology, travel, history, and more. Each query is tagged with a category label. These questions are then converted into high-dimensional numerical vectors using a sentence embedding model, enabling analysis of their semantic relationships. By applying clustering and classification techniques to the embedded data, the system can organize queries into meaningful groups and accurately assign category labels - even for previously unseen inputs.

To achieve this, the system uses SBERT sentence embedding model to convert queries into vector form, applies k-means clustering to group semantically similar queries, and finally uses a multinomial logistic regression classifier to assign category labels to input queries.

## Datasets

To generate the main dataset used in the project, a custom prompt is sent to the Deepseek V3 model via the OpenRouter API. This prompt requests Deepseek to generate several hundred diverse, natural-language questions that a typical end user might ask a large language model. The questions are organized into generalized categories such as technology, travel, education, or history to simulate a realistic range of LLM use cases. The prompt also enforces a strict response format: each question must be returned as a JSON object with two keys—"Question" and "Category". This structure ensures consistency in downstream parsing and processing.

The dataset generation is done in a separate code file before the embedding and clustering step.

A prompt is loaded from a text file and modified to include the desired number of queries.

Deepseek is prompted to return the questions in a JSON format with both the question and category. The `generate_queries()` function then uses the Deepseek model to synthesize the data. Once a successful response is returned, the user is prompted to confirm it for correctness before the dataset is saved.

## **Embedding**

Embedding the queries is a necessary step in the processing of natural language data to assign numerical values to semantic material. These embeddings can then be used to further process, modify, or classify the natural language data. For the purposes of this project, we have chosen to use the Siamese Bidirectional Encoder Representations from Transformers (SBERT) model. This model is specifically designed to encode natural language data in a way which preserves semantic context/meaning within the sentence. SBERT was chosen as it is specialized for providing highly accurate semantic similarity scores when compared to competing models (e.g., BERT). Embedding the natural language data into matrices using SBERT preprocesses and prepares the data for the clustering process.

## **Clustering**

Clustering uses the numerical embeddings provided by SBERT to group natural language data into semantically similar groupings called clusters. The goal of the clustering process is to preprocess the data for category classification and to establish which queries should be grouped together under the same categories. A k-means clustering algorithm was chosen for this use case as it is widely used for the purpose of grouping together semantically similar natural language data, is simple to implement, and works very quickly. The k-means clustering algorithm functions by initially placing a set number of cluster centroids in the vector space randomly, then assigning each of the data vectors to their nearest cluster centroids. The algorithm then shifts the centroids by calculating the mean of the data points which were assigned to each of the centroids. Finally, the process is repeated until no significant change occurs. This also means that the clustering algorithm is highly dependent on the initial placement of the cluster centroids.

A fundamental limitation of the k-means algorithm is that it requires pre-specifying the number of clusters  $k$ . In practical applications, especially when dealing with natural language data, we rarely know in advance exactly how many semantic groups the data should be divided into. Choosing an inappropriate  $k$  value can lead to serious problems: too small  $k$  value results in different semantic concepts being incorrectly merged into the same cluster; too large  $k$  value artificially splits data points that should belong to the same concept. To mitigate this disadvantage, a silhouette score is calculated for each of the results of the algorithm running on

up to two times the number of categories provided in the dataset. The silhouette score measures how close each of the data points in the cluster is to the others as well as the separation between each point in the cluster to neighboring clusters. The model with the highest silhouette score is picked as the ‘best’ and its centroids are used for the final classification to assign each cluster their corresponding category labels.

## **Classification**

After clustering the queries, the next step was to predict categories for new, unseen questions. A multinomial logistic regression model was chosen for this task using the same SBERT embeddings from the clustering step. This approach kept the pipeline simple, quick, and reproducible. It suited the project’s needs due to its fast training on CPU, easy to understand coefficients, and proven effectiveness, often performing as well as more complex neural networks.

Before training, queries retained their original LLM-generated labels, resulting in clean and L2-normalized vectors. A grid search optimized two hyperparameters, the inverse regularization parameter C (tested from 0.001 to 1000) and class weighting (none or balanced). Stratified cross-validation focusing on the macro F1 score selected a C value of 1.0 without class balancing. The final model achieved an impressive accuracy of  $0.958 \pm 0.023$ , with a macro precision of  $0.967 \pm 0.016$ , a macro recall of  $0.958 \pm 0.025$ , and a macro F1 score of  $0.958 \pm 0.024$ . Most errors occurred between similar categories, such as Creative Writing and Content Creation, or Math/Logic and Philosophy. When tested on a separate test dataset, the model maintained strong performance with an accuracy of 0.955, confirming its ability to generalize to unseen queries.

Although the linear model may struggle with subtle, complex boundaries between conceptually related categories, its simplicity, speed, and transparency make it ideal for the project's current scope. The confusion matrix analysis revealed near-perfect classification for several categories, including Data Analysis, Education, and Health/Wellness. Future improvements could involve testing kernel SVMs or adding more data for categories showing the most confusion.

## Why this Pipeline Works

K-means clustering assumes that data points lie in a continuous vector space and that meaningful groupings can be formed by minimizing intra-cluster Euclidean distance. SBERT generates fixed-size, dense, real-valued sentence embeddings, satisfying this requirement by mapping semantically similar sentences to nearby points in high-dimensional space.

While SBERT is trained with cosine similarity objectives, cosine and Euclidean distances become equivalent once vectors are L2-normalized. Normalizing SBERT embeddings thus makes Euclidean distance a valid proxy for semantic similarity, aligning them with the assumptions of k-means.

Although k-means works best on spherical, equally sized clusters, SBERT embeddings often form locally compact and roughly isotropic regions. These natural structures make them reasonably well-suited for clustering with k-means.

In practice, this alignment enables k-means to group semantically related queries based on SBERT embeddings, making their combination a practical strategy for unsupervised semantic clustering.

## Conclusion

The Query Classifier project was successful in exploring prompt generation using a large language model API, embedding queries, clustering queries by semantic similarity, and in using the cluster centers to classify each cluster with a category label. The project explored dataset generation using a large language model (LLM) and was able to create a dataset of questions someone might ask an LLM for assistance or feedback. This dataset was used to test the efficacy of classifying queries into different groups with category labels. The dataset was first embedded using SBERT to transform into a numerical representation, which could be used as the input to a k-means clustering algorithm to group similar queries. The dataset queries and category labels were then used to train a logistic regression classifier that achieved strong performance metrics, with cross-validation showing 95.8% accuracy ( $\pm 2.3\%$ ) and an F1 score of 0.958 ( $\pm 0.024$ ). When tested on unseen data, the model maintained robust performance with 95.5% accuracy, demonstrating excellent generalization capabilities. Finally, the cluster centers were fed into the

classifier to return category labels for each cluster of queries, effectively organizing the diverse range of user questions into meaningful semantic categories.